Bit	7	6	5	4	3	2	1	0
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00
Read/Write	W	RW	RW	RW	RW	RW	RW	RW
Initial Value	0	0	0	0	0	0	0	0

FOC0 D7 Force Output compare: Writing 1 to it forces the wave generator to act as if a compare match has occurred.

WGM01:00 D6 D3 Timer0 mode selector bits

D6	D3	
0	0	Normal
0	1	CTC (Clear Timer on Compare match)
1	0	PWM, phase correct
1	1	Fast PWM

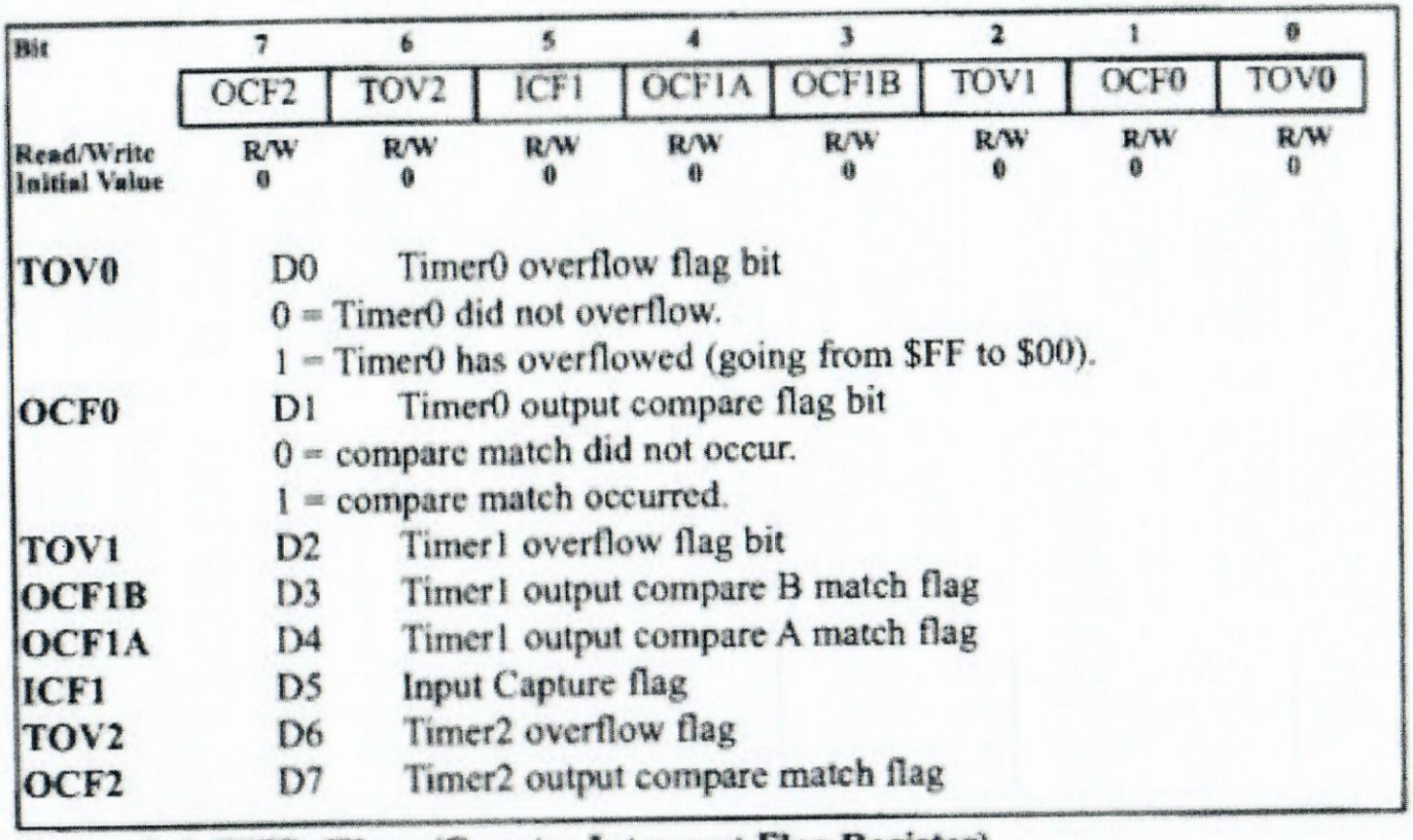
COM01:00 D5 D4 Compare Output Mode; The table shows what the wave generator does on compare match when the timer is in Normal or CTC mode:

COM01	COM00	Description
0	0	Normal port operation, OC0 disconnected
0	1	Toggle OC0 on compare match
1	0	Clear OC0 on compare match
1	1	Set OC0 on compare match

CS02:00 D2D1D0 Timer0 clock selector

D2	D1	D0	
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	clk (no prescaling)
0	1	0	clk / 8
0	1	1	clk / 64
1	0	0	clk / 256
1	0	1	clk / 1024
1	1	0	External clock source on T0 pin. Clock on falling edge
1	1	1	External clock source on T0 pin. Clock on rising edge

Figure 15-4. TCCR0 (Timer/Counter Control Register) Register



Bit	7	6	5	4	3	2	1	0
	OCF2	TOV2	ICF1	OCF1A	OCF1B	TOV1	OCF0	TOV0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

TOV0 D0 Timer0 overflow flag bit
0 = Timer0 did not overflow.
1 = Timer0 has overflowed (going from $FF to $00).

OCF0 D1 Timer0 output compare flag bit
0 = compare match did not occur.
1 = compare match occurred.

TOV1 D2 Timer1 overflow flag bit

OCF1B D3 Timer1 output compare B match flag

OCF1A D4 Timer1 output compare A match flag

ICF1 D5 Input Capture flag

TOV2 D6 Timer2 overflow flag

OCF2 D7 Timer2 output compare match flag

Figure 9-6. TIFR (Timer/Counter Interrupt Flag Register)

Bit	7	6	5	4	3	2	1	0
	COM1A1	COM1A0	COM1B1	COM1B0	FOC1A	FOC1B	WGM11	WGM10
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

COM1A1:COM1A0 D7 D6 Compare Output Mode for Channel A (discussed in Section 9-3)

COM1B1:COM1B0 D5 D4 Compare Output Mode for Channel B (discussed in Section 9-3)

FOC1A D3 Force Output Compare for Channel A (discussed in Section 9-3)

FOC1B D2 Force Output Compare for Channel B (discussed in Section 9-3)

WGM11:10 D1 D0 Timer1 mode (discussed in Figure 9-18)

Figure 9-17. TCCR1A (Timer 1 Control) Register

Bit	7	6	5	4	3	2	1	0	
	ICNC1	ICES1	-	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ICNC1 D7 Input Capture Noise Canceler
0 = Input Capture is disabled.
1 = Input Capture is enabled.

ICES1 D6 Input Capture Edge Select
0 = Capture on the falling (negative) edge
1 = Capture on the rising (positive) edge

D5 Not used

WGM13:WGM12 D4 D3 Timer1 mode

Mode	WGM13	WGM12	WGM11	WGM10	Timer/Counter Mode of Operation	Top	Update of OCR1x	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	TOP	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	TOP	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	TOP	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	Reserved	-	-	-
14	1	1	1	0	Fast PWM	ICR1	TOP	TOP
15	1	1	1	1	Fast PWM	OCR1A	TOP	TOP

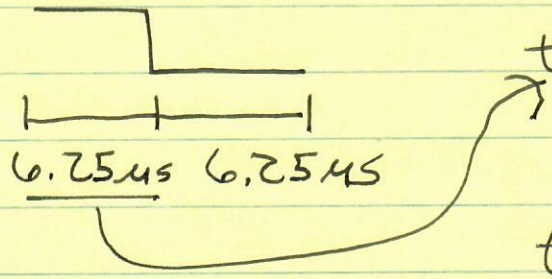
CS12:CS10 D2D1D0 Timer1 clock selector

0	0	0	No clock source (Timer/Counter stopped)
0	0	1	clk (no prescaling)
0	1	0	clk / 8
0	1	1	clk / 64
1	0	0	clk / 256
1	0	1	clk / 1024
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

Figure 9-18. TCCR1B (Timer 1 Control) Register

Assume $f_{osc} = 8\,MHz$. Write a program to generate an 80kHz square wave on PORTC bit 2 (RC2). Timer 0

$f = 80kHz \Rightarrow T = \frac{1}{f} = \frac{1}{80(10^3)} = 12.5\mu s$



$t_{delay} = n t_{MC}$

$t_{MC} = \left(\frac{1\,cc}{TMc}\right)\left(\frac{1s}{8(10^6)\,cc}\right)$

$= 0.125\mu s/MC$

$n = \frac{6.25\mu s}{0.125\mu s} = 50$ ticks

TCNTO $= -50$ or $256 - 50 = 206$

hex

CE

```
;  in class example - 80 kHz square wave


.ORG 0
LDI R16, HIGH(RAMEND)
OUT SPH, R16
LDI R16, LOW(RAMEND)
OUT SPL, R16; MOVE SP BECAUSE USING SUBROUTINE

SBI DDRC, 2; PC2 OUTPUT
LDI R17, 0X04; 0B00000100 BIT POSITION 2 SET
LDI R18, 0
OUT PORTC, R18; INITIALIZE PC2 = 0

BEGIN:      RCALL DELAY
            EOR R18, R17; TOGGLE PC2
            OUT PORTC, R18; UPDATE PC2
            RJMP BEGIN

            ; TIMER 0 DELAY
DELAY:      LDI R21, -50; PRELOAD FOR NUMBER OF TICKS FOR HALF WAVE
            OUT TCNT0, R21
            LDI R21, 0X01; 0B00000001-LAST 3 BITS 001=> CLK NO PRESCALER
            OUT TCCR0, R21
AGAIN:      IN R21, TIFR
            SBRS R21, TOV0; CHECK STATUS OF TOV0
            RJMP AGAIN; WAIT FOR TIMER0 TO OVERFLOW (TOV0 SET)
            LDI R21, 0X00
            OUT TCCR0, R21; TURN OFF TIMER 0
            LDI R21, (1<<TOV0); RESET TOV0 (WRITE A 1 TO IT, 0S TO OTHER BITS)
            OUT TIFR, R21
            RET
```

```
DELAY MACHINE CYCLES

;  in class example - 80 kHz square wave


.ORG 0
LDI R16, HIGH(RAMEND)
OUT SPH, R16
LDI R16, LOW(RAMEND)
OUT SPL, R16; MOVE SP BECAUSE USING SUBROUTINE

SBI DDRC, 2; PC2 OUTPUT
LDI R17, 0X04; 0B00000100 BIT POSITION 2 SET
LDI R18, 0
OUT PORTC, R18; INITIALIZE PC2 = 0

BEGIN:     RCALL DELAY
           EOR R18, R17; TOGGLE PC2
           OUT PORTC, R18; UPDATE PC2
           RJMP BEGIN

      MCs   ; TIMER 0 DELAY
DELAY: 1    LDI R21, -50; PRELOAD FOR NUMBER OF TICKS FOR HALF WAVE
       1    OUT TCNT0, R21
       1    LDI R21, 0X01; 0B00000001-LAST 3 BITS 001=> CLK NO PRESCALER
       1    OUT TCCR0, R21
AGAIN:  1   IN R21, TIFR
       2/1  SBRS R21, TOV0; CHECK STATUS OF TOV0
       2    RJMP AGAIN; WAIT FOR TIMER0 TO OVERFLOW (TOV0 SET)
       1    LDI R21, 0X00
       1    OUT TCCR0, R21; TURN OFF TIMER 0
       1    LDI R21, (1<<TOV0); RESET TOV0 (WRITE A 1 TO IT, 0S TO OTHER BITS)
       1    OUT TIFR, R21
       4    RET
```

DELAY MACHINE CYCLES => 1 LDI + 1 OUT + 1 LDI + 1 OUT (ONLY EXECUTED ONCE)
+ (AGAIN LOOP) (1 IN + 1 SBRS + 2 RJMP) 13 TIMES + 1 IN + 2 SBRS
+ 1 LDI + 1 OUT+ 1 LDI + 1 OUT + 4 RET (INSTRUCTIONS AFTER AGAIN LOOP)
= 4+[13(4)+3]+8 = 67 MCS (ACTUAL COMPARED TO TARGETED 50 MCS FOR TIMER0)
ERROR = 17/50 = 34%

AGAIN LOOP EXPLANATION

(12 TIMES TOV0 NOT SET, IN 13TH ITERATION TOV0 NOT SET WHEN IN INSTRUCTION EXECUTED, SO MUST COMPLETE 13TH ITERATION) AND THE FIRST 2 INSTRUCTIONS (IN, SBRS FOR 14TH ITERATION SO THAT TOV0 IS DETECTED)TO GET 50 MCS THEN MUST GO THROUGH LOOP AGAIN BECAUSE TOV0 SET BUT NOT DETECTED IN SBRS INSTRUCTION YET SO + 1 IN + 1 SBRS

```
;  in class example - 80 kHz square wave
; CTC MODE

.ORG 0
LDI R16, HIGH(RAMEND)
OUT SPH, R16
LDI R16, LOW(RAMEND)
OUT SPL, R16; MOVE SP BECAUSE USING SUBROUTINE

SBI DDRC, 2; PC2 OUTPUT
LDI R17, 0X04; 0B00000100 BIT POSITION 2 SET
LDI R18, 0
OUT PORTC, R18; INITIALIZE PC2 = 0
OUT TCNT0, R18; INITIALIZE TCNT0 = 0, RESETS ITSELF WHEN OCF0 IS SET

LDI R19, 49; CTC => COUNT 0 TO 50-1=49
OUT OCR0, R19; INITIALIZE OUTPUT COMPARE REGISTER FOR TIMER 0



BEGIN:      RCALL DELAY
            EOR R18, R17; TOGGLE PC2
            OUT PORTC, R18; UPDATE PC2
            RJMP BEGIN

            ; TIMER 0 DELAY
DELAY:      LDI R21, 0X09; 0B00001001- BITS 6 AND 3=>WGM01:00 = 01 FOR CTC MODE
                        ;LAST 3 BITS 001=> CLK NO PRESCALER
            OUT TCCR0, R21
AGAIN:      IN R21, TIFR
            SBRS R21, OCF0; CHECK STATUS OF OUTPUT COMPARE MATCH FOR TIMER 0 OCF0
            RJMP AGAIN; WAIT FOR TIMER0 TO OVERFLOW (TOV0 SET)
            LDI R21, 0X00
            OUT TCCR0, R21; TURN OFF TIMER 0
            LDI R21, (1<<OCF0); RESET OCF0 (WRITE A 1 TO IT, 0S TO OTHER BITS)
            OUT TIFR, R21
            RET
```

```
/*
 * Timer0CExample.c


 */

#include <avr/io.h>
#define preload -150

void delayroutine();

int main(void)
{
    DDRB = DDRB | 0b00100000; // pb5 output

    while (1)
    {
        PORTB = PORTB ^ 0b00100000; // pb5 = 1
        delayroutine();
    }
    return 0;
}
void delayroutine()
{
    TCNT0 = preload;
    TCCR0 = 0b00000001;
    while (!(TIFR & (1<<TOV0)));
    TCCR0 = 0x00;
    TIFR = 1<<TOV0;

}
```

This program is 50% duty cycle. TCNT0 = -150

```c
/*
 * Timer0CExample.c
 *
 * Author : stanleyj
 */

#include <avr/io.h>
#define preload -150

void delayroutine();

int main(void)
{
    DDRB = DDRB | 0b00100000; // set pb5

    while (1)
    {   // 50% duty cycle
            PORTB = PORTB ^ 0b00100000; // toggle pb5
            delayroutine();

    }
       return 0;
}

void delayroutine()
{
       TCNT0 = preload;
       TCCR0 = 0b00000001;
       while (!(TIFR & (1<<TOV0))); // polling
       TCCR0 = 0x00;
       TIFR = 1<<TOV0;

}

//////////////////////////////////////////////////////////////////////
#include <avr/io.h>
#define preload -150

void delayroutine();

int main(void)
{
       DDRB = DDRB | 0b00100000; // set pb5

       while (1)
       {      // 66% duty cycle
              PORTB |= 0b00100000; // set pb5
              delayroutine();
              delayroutine();
              PORTB &= 0b11011111; // clear pb5 OR PORTB = PORTB ^ 0b00100000;
              delayroutine();
       }
       return 0;
}

void delayroutine()
{
       TCNT0 = preload;
       TCCR0 = 0b00000001;
       while (!(TIFR & (1<<TOV0))); // polling
       TCCR0 = 0x00;
       TIFR = 1<<TOV0;

}
```